INSIGNIA

AWS Business Unit

Data Engineer

Take-Home Assessment

Level - 2

Prepared by: **Data Engineering Team**
AWS Business Unit

Version 2.0 | January 2026

General Instructions

Welcome to the Insignia Data Engineer technical assessment. This take-home test evaluates your practical skills across three core competencies: data pipeline development (Python/PySpark), SQL query design and optimization, and data architecture and system design.

Important: Read the scenario carefully before starting — the context you build in Question 1 carries through to Questions 2 and 3.

Evaluation Criteria

Pipeline Design: Clean, testable, production-ready ETL/ELT code with proper error handling and logging.

SQL Proficiency: Correct, performant queries demonstrating window functions, CTEs, and optimization awareness.

Architecture Thinking: Understanding of data warehouse design, medallion architecture, scalability, and trade-offs.

Explanation: Clear reasoning behind your approach. We value your thought process as much as the solution.

LEVEL 2 — DATA ENGINEER

This level tests deeper architectural thinking and production engineering skills expected of a mid-level data engineer. All three questions are set within the same company and dataset — read the scenario first.

📖 **THE SCENARIO — NegaraBank**

NegaraBank is an Indonesian digital-first bank with 4.2 million customers, offering savings accounts, personal loans, credit cards, QRIS payments, and investment products (mutual funds and government bonds). They operate entirely through a mobile app and web platform — no physical branches.

Their data infrastructure includes: a core banking system (Oracle) processing 2M+ transactions/day, a mobile app generating clickstream events via Kafka (50K events/sec at peak), a credit scoring ML pipeline on Databricks, a customer service platform (Zendesk) with 15K tickets/month, and marketing campaign data from Braze and Google Ads.

NegaraBank recently received a directive from OJK (Indonesia's financial regulator) requiring them to demonstrate data lineage and auditability for all customer-facing metrics within 6 months. The CDO has assembled a cross-functional data team: data engineers (you), BI analysts, and data governance analysts all working on the same platform.

You've been hired as the mid-level data engineer to fix and optimize a critical ELT pipeline (Q1), build complex analytical queries for regulatory reporting (Q2), and design the end-to-end data platform architecture (Q3).

Shared Database Schema

The following tables exist across NegaraBank's data warehouse. All roles work with the same dataset.

`bronze.accounts`

Column	Type	Description
account_id	VARCHAR(20)	Unique account identifier
customer_id	VARCHAR(15)	FK to customers
account_type	VARCHAR(20)	SAVINGS, LOAN, CREDIT_CARD, INVESTMENT
product_name	VARCHAR(50)	Specific product (e.g., Tabungan Negara Plus)
opened_date	DATE	Account opening date
status	VARCHAR(15)	ACTIVE, DORMANT, CLOSED, SUSPENDED

balance	DECIMAL(15,2)	Current balance (snapshot)
credit_limit	DECIMAL(15,2)	For credit cards only, else NULL
interest_rate	DECIMAL(5,4)	Annual interest rate

bronze.transactions

Column	Type	Description
txn_id	VARCHAR(25)	Unique transaction ID
account_id	VARCHAR(20)	FK to accounts
txn_date	TIMESTAMP	Transaction timestamp
txn_type	VARCHAR(20)	DEBIT, CREDIT, TRANSFER_IN, TRANSFER_OUT, PAYMENT, FEE
amount	DECIMAL(15,2)	Transaction amount
merchant_category	VARCHAR(30)	MCC category (nullable)
channel	VARCHAR(20)	MOBILE_APP, WEB, QRIS, ATM_NETWORK, AUTO_DEBIT
reference_id	VARCHAR(30)	External reference (nullable)
status	VARCHAR(15)	COMPLETED, PENDING, FAILED, REVERSED

bronze.customers

Column	Type	Description
customer_id	VARCHAR(15)	Unique customer identifier
full_name	VARCHAR(100)	Customer legal name
nik	VARCHAR(16)	National ID number (KTP)
phone	VARCHAR(15)	Primary phone
email	VARCHAR(80)	Email address
city	VARCHAR(30)	City of residence
province	VARCHAR(30)	Province
registration_date	DATE	Onboarding date
kyc_status	VARCHAR(15)	VERIFIED, PENDING, REJECTED
risk_score	DECIMAL(5,2)	ML-generated risk score (0–100)
segment	VARCHAR(20)	MASS, EMERGING_AFFLUENT, AFFLUENT, HIGH_NET_WORTH

bronze.credit_scores

Column	Type	Description
score_id	VARCHAR(20)	Unique score record
customer_id	VARCHAR(15)	FK to customers
score_date	DATE	Date score was generated
model_version	VARCHAR(10)	ML model version (v2.1, v2.2, etc.)
credit_score	INTEGER	Score 300–850
probability_of_default	DECIMAL(5,4)	PD estimate (0.0000–1.0000)
features_used	JSONB	Feature importance JSON

bronze.app_events

Column	Type	Description
event_id	VARCHAR(30)	Unique event ID
customer_id	VARCHAR(15)	FK to customers
event_timestamp	TIMESTAMP	When the event occurred
event_type	VARCHAR(30)	screen_view, button_click, transaction_initiated, etc.
screen_name	VARCHAR(50)	App screen name
session_id	VARCHAR(30)	Session identifier
device_type	VARCHAR(15)	ANDROID, IOS, WEB
app_version	VARCHAR(10)	App version string

bronze.support_tickets

Column	Type	Description
ticket_id	VARCHAR(15)	Zendesk ticket ID
customer_id	VARCHAR(15)	FK to customers
created_at	TIMESTAMP	Ticket creation time
resolved_at	TIMESTAMP	Resolution time (nullable)
category	VARCHAR(30)	TRANSACTION, ACCOUNT, CARD, LOAN, GENERAL
priority	VARCHAR(10)	LOW, MEDIUM, HIGH, URGENT
satisfaction_score	INTEGER	CSAT 1–5 (nullable)
channel	VARCHAR(15)	IN_APP, EMAIL, CALL, SOCIAL_MEDIA

Question 1: Pipeline Optimization — Fixing the Daily Account Snapshot ELT

Python/PySpark, Databricks/Snowflake, Incremental Loading

Context

NegaraBank runs a nightly ELT job that creates a daily account snapshot in the Silver layer. The job is critical — it feeds the BI dashboards, the credit scoring pipeline, and the regulatory reporting views. But it's been failing intermittently, takes 4+ hours to run (for only 8M rows), and produces inconsistent results. Here's the current code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("account_snapshot").getOrCreate()

accounts = spark.read.format("jdbc").option("url", "jdbc:oracle:thin:@core-
db:1521/banking") \
    .option("dbtable", "ACCOUNTS").option("user", "etl_user").option("password",
"etl_pass123").load()

txns = spark.read.format("jdbc").option("url", "jdbc:oracle:thin:@core-
db:1521/banking") \
    .option("dbtable", "TRANSACTIONS").load()
today_txns = txns.filter(txns.txn_date >= "2026-01-01")

snapshot = accounts.join(today_txns, "account_id", "left") \
    .groupBy("account_id", "customer_id", "account_type", "balance") \
    .agg({"amount": "sum", "txn_id": "count"})

snapshot.write.mode("overwrite").parquet("s3://negarabank-
silver/account_snapshots/")
```

Tasks

a. Identify at least 6 problems with this pipeline (performance, correctness, security, reliability). For each, explain the production impact and propose the fix.

b. Rewrite this pipeline as production-grade PySpark (or Snowflake SQL if you prefer). Your solution must: use incremental loading (don't read all transactions every night), partition the output by snapshot_date for time-travel queries, include proper secret management (no hardcoded credentials), add data quality assertions (e.g., row count sanity check, balance reconciliation), and be idempotent (re-running for the same date produces the same result).

c. The credit scoring team needs to consume this snapshot within 30 minutes of it being ready (current SLA is 'sometime before 8 AM'). Design a notification/orchestration mechanism using Airflow, Databricks Workflows, or Step Functions that: triggers the credit scoring pipeline after the snapshot succeeds, retries on failure and alerts the engineer if it fails 3 times.

Question 2: Advanced SQL — Regulatory Reporting Views

SQL (PostgreSQL), CTEs, Window Functions, Optimization

Context

OJK requires NegaraBank to submit monthly regulatory reports with specific metrics. The BI team has built dashboards, but they need you to build the underlying performant SQL views that power them. These views will be materialized nightly and must handle the full dataset (8M accounts, 60M+ transactions).

Tasks

d. Write a SQL query for the Monthly Customer Health Scorecard. For each customer, calculate: total balance across all accounts, month-over-month balance change (using LAG), number of transactions by type (DEBIT, CREDIT, etc.) using PIVOT or conditional aggregation, average transaction amount per channel, credit utilization ratio (balance / credit_limit for credit cards), and a risk_flag: TRUE if credit_utilization > 80% OR probability_of_default > 0.3 OR balance declined > 30% MoM. The query should be optimized.

e. Write a SQL query that detects potentially fraudulent transaction patterns. Flag customers who: made 5+ transactions within a 1-hour window (use window functions with RANGE/ROWS), have transactions in 3+ different cities on the same day (requiring a join to merchant location data), or have a single transaction exceeding 3x their 30-day average transaction amount. Produce an output with customer_id, alert_type, alert_date, details_json.

f. The current nightly materialization of these views takes 45 minutes. Propose an optimization strategy: would you use Snowflake dynamic tables, dbt incremental models, or materialized views? Justify your choice. Write the DDL for one of the views above, showing how incremental refresh would work.

Question 3: Platform Architecture — NegaraBank Data Platform Design

AWS, Snowflake/Databricks, Kafka, System Design

Context

The CDO wants you to design NegaraBank's end-to-end data platform architecture. It must support: the batch ELT pipelines from Q1, the regulatory reporting views from Q2, real-time clickstream analytics from the mobile app (50K events/sec), the credit scoring ML pipeline, and the data governance requirements the DGQA team is implementing (lineage, cataloging, access control). You have a $50K/month cloud budget.

Tasks

g. Draw or describe the complete architecture diagram. Show: data sources (Oracle, Kafka, Zendesk, Braze), ingestion layer (how batch and streaming data enter the platform), storage layers (Bronze/Silver/Gold with specific technologies), processing engines (Spark, Snowflake, etc.), serving layer (how BI, ML, and regulatory consumers access data), and governance layer (where data catalog, lineage, and access control fit). For each component, justify why you chose it over alternatives.

h. The mobile app generates 50K clickstream events per second at peak. Design the real-time ingestion pipeline from Kafka to an analytics-ready state. How you would make this data joinable with the batch account data from Q1, and the hot/warm/cold storage strategy for event data (recent vs. historical).

i. The DGQA team needs column-level lineage from source (Oracle) to consumption (BI dashboard). How would you instrument your pipelines to enable this? How would this integrate with the pipeline you wrote in Q1?